MODULE 2 · SW DESIGN

Fundamentals of Software Design

The two forces behind every good design — and a law that keeps them honest.

Akın Kaldıroğlu · Lead Trainer

RECAP & TODAY

From 'What' to 'How'

Last time: design is decision-making, and it lives on a continuum with architecture. Today we get concrete tools to judge those decisions.

Cohesion

Does each part do one thing well?

Coupling

How tangled are the parts with each other?

Change

Will the next requirement be cheap — or painful?

WHAT YOU'LL LEAVE WITH

Learning Objectives

- Define cohesion and coupling — and recognise them in real code.
- Explain why 'high cohesion, low coupling' is the golden rule of design.
- See how coupling turns a small change into an expensive one.
- Apply the Law of Demeter to tame dependency chains.

SECTION 01

Cohesion

How well the things inside a module belong together.

THE BIG PICTURE

Two Forces Shape Every Design

Almost everything we call 'good design' comes down to balancing two measurable forces. Keep them in mind for the rest of the bootcamp:

Cohesion (want HIGH)

- How focused a single module is
- One clear responsibility
- Measured within a module

Coupling (want LOW)

- How dependent modules are on each other
- How much one must know about another
- Measured between modules

DEFINITION

What Is Cohesion?

Cohesion is the degree to which the elements inside a module belong together — how single-minded that module is about its job.

High cohesion = a class or function that does **one thing**, and everything in it serves that thing.

SEE IT IN CODE

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.cohesion`

Low vs High Cohesion

A 'God' utility that does unrelated jobs vs. focused classes with one responsibility each:

Java *One class, four reasons to change*

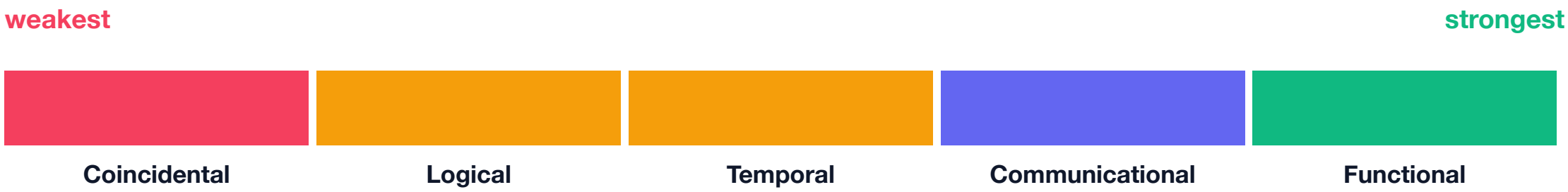
```java
// LOW cohesion — unrelated responsibilities in one class
class Utils {
    void saveUser(User u) { /* database */ }
    void sendEmail(String to) { /* SMTP */ }
    double calcTax(Order o) { /* finance */ }
    String formatDate(Date d) { /* formatting */ }
}
```

Split into **UserRepository**, **EmailService**, **TaxCalculator** — each cohesive, each with a single reason to change.

A USEFUL SCALE

Cohesion Comes in Degrees

Not all cohesion is equal. Aim for the right end of this scale:



Functional cohesion — every element contributes to a single, well-defined task — is the goal. You don't need the jargon; you need the instinct.

SECTION 02

Coupling

How much modules depend on — and know about — each other.

DEFINITION

What Is Coupling?

Coupling is the strength of the dependencies between modules — how much one module must know about another to work.

Loose coupling = modules interact through small, stable interfaces and know as **little** about each other as possible.

SEE IT IN CODE

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.coupling`

Tight vs Loose Coupling

Depending on a concrete class welds you to it. Depending on an interface lets you swap the other side freely:

Java — tight

```
class Report {
  MySqlDb db =
     new MySqlDb();
  // welded to MySQL
  void run() {
    db.query(...);
  }
}
```

Java — loose

```
class Report {
  final Database db;
  Report(Database db){
    this.db = db;   // any DB
  }
  void run() {
    db.query(...);
  }
}
```

The loose version depends on an abstraction — swap MySQL for Postgres or a fake in tests, no change to Report.

WHY IT MATTERS

Coupling Is a Change Multiplier

The cost of coupling shows up the day you change something. Tightly coupled code spreads one change across many files:

Tightly coupled

- One change ripples outward
- Hard to test a part in isolation
- Fear of touching anything

Loosely coupled

- Changes stay local
- Parts tested independently
- Confidence to evolve

THE GOLDEN RULE

High Cohesion, Low Coupling

Group what changes together. Separate what changes apart.

High cohesion pulls together

- Related behaviour lives in one place
- Easy to find and understand

Low coupling keeps apart

- Unrelated parts stay independent
- Easy to change one without the others

SECTION 03

Designing for Change

Cohesion and coupling only matter because software changes.

THE REAL GOAL

We Optimise for Change

You will never predict every future requirement. So you don't design to predict change — you design to absorb it cheaply when it comes.

- Isolate what varies — put likely changes behind a boundary.
- Hide decisions — callers shouldn't depend on how something works.
- Depend on abstractions, not details (we'll formalise this in SOLID).
- Small, cohesive, loosely coupled parts = a system that bends, not breaks.

THE MAIN TOOL

Encapsulation & Information Hiding

Encapsulation is how we buy low coupling: expose a small, stable surface; hide the volatile details behind it.

Expose (stable)

- What the module does
- A small, intention-revealing API
- Types the caller truly needs

Hide (volatile)

- How it does it — algorithms, data
- Internal fields & helper types
- Third-party libraries & storage

SECTION 04

The Law of Demeter

A simple rule to stop coupling from creeping back in.

THE PRINCIPLE

Don't Talk to Strangers

The Law of Demeter (the 'principle of least knowledge') says a method should only talk to its immediate friends — not reach through them to strangers.

- Only call methods on: itself, its fields, its parameters, objects it creates.
- Avoid long chains: `a.getB().getC().doSomething()` reaches through others.
- Each dot past the first is a new thing you've secretly coupled to.

THE SMELL

▶ runnable `dev.kaldiroglu.bootcamp.fundamentals.demeter`

The 'Train Wreck'

This line knows the shape of Order, Customer AND Address. Change any one and it may break:

Java

```
String city = order
  .getCustomer()
  .getAddress()
  .getCity();
// reaches 3 classes
```

C#

```
var city = order
  .Customer
  .Address
  .City;
// reaches 3 classes
```

Python

```
city = (order
  .customer
  .address
  .city)
# reaches 3 classes
```

Every getter in the chain is a dependency in disguise.

THE FIX

▶ runnable dev.kaldiroglu.bootcamp.fundamentals.demeter

Tell, Don't Ask

Ask the object you know to do the job — let it deal with its own internals:

Java

The chain is gone — Order hides its structure

```
// Instead of reaching through order for the city...
String city = order.shippingCity();   // ask the object you have

class Order {
    String shippingCity() {
        return customer.shippingCity();   // each object knows its own parts
    }
}
```

Now callers depend only on Order. Its internal shape is free to change.

BALANCE

Powerful — but Not Absolute

It buys you

- Lower coupling between classes
- Freedom to change internals
- Code that reads as intent, not plumbing

Keep perspective

- It's a heuristic, not a hard law
- Fluent builders & streams are fine
- Aim for less knowledge, not zero dots

REMEMBER THESE

Key Takeaways

- ✓ Cohesion (high) and coupling (low) are the two forces of good design.
- ✓ Cohesion: one module, one job. Coupling: know as little as possible.
- ✓ Coupling is a change multiplier — it turns small edits into big ones.
- ✓ Encapsulation buys low coupling: expose a stable surface, hide the details.
- ✓ Law of Demeter: talk to friends, not strangers — kill the train wrecks.

NEXT SESSION

What's Next

MODULE 3 · SATURDAY 18 JULY

Clean Code & SOLID

with Akın Kaldıroğlu

Turning these forces into everyday habits: a Clean Code framework, then the five SOLID principles that make high cohesion and low coupling systematic.

FURTHER PRACTICE

Find These Smells for Real

Every force from this session lives in the ACME Orders codebase. Measure the cohesion and coupling, then improve them.

Refactoring course · Cohesion & Coupling

Repo `github.com/kaldiroglu/refactoring-course-student-materials`

IN THIS SESSION, LOOK AT

- `Exercises/Ex01-CohesionCoupling.md`
- `.../service/OrderService.java  (coupled god-service)`
- `.../util/Util.java  (low cohesion)`

Questions?

High cohesion, low coupling — everything else is detail.